

บทที่ 4

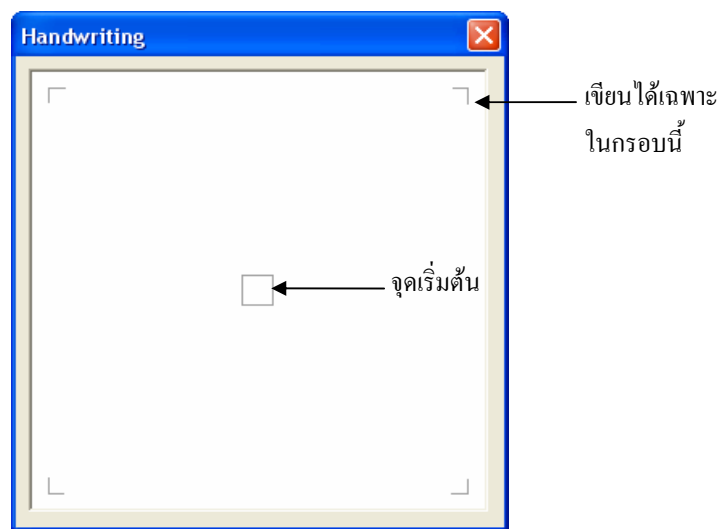
การออกแบบและพัฒนา

4.1 การหาวิธีรับลายมือเขียนผ่านทางการลากเมาส์โดยไม่ต้องคลิกเมาส์

จากการที่เราต้องการหาวิธีรับลายมือเขียนโดยผ่านทางการลากเมาส์โดยไม่ต้องคลิกเมาส์ เนื่องจากการคาดหวังว่าเมาส์ซึ่งเป็นอุปกรณ์ที่เครื่องคอมพิวเตอร์ทุกเครื่องในปัจจุบันจำเป็นต้องมีจะสามารถนำมาใช้เขียนตัวอักษรแทนการใช้ปากกาได้ ซึ่งวิธีการเดิมที่ใช้กันมาจะใช้การแฉีกเมาส์ (การกดเมาส์แล้วลาก) แต่การที่ต้องกดปุ่มของเมาส์ขณะลากไปด้วยทำให้เขียนลำบาก ขาดความคล่องตัว ทำให้เราคิดว่าน่าจะลองหาวิธีที่ไม่ต้องคลิกเมาส์ดู

เราได้ทำการศึกษาหาวิธีการรับลายมือเขียนที่คิดว่าเป็นวิธีที่เหมาะสม โดยมุ่งเน้นไปที่การใช้เมาส์ลากโดยไม่ต้องคลิกเมาส์ ซึ่งการที่ไม่ต้องคลิกเมาส์นี้ทำให้การลากเมาส์เป็นไปอย่างอิสระและสามารถเขียนได้เร็วกว่าการที่ต้องคลิกเมาส์อีกด้วย

วิธีการที่ได้ใช้ในตอนแรกก็คือได้กำหนดจุดจุดหนึ่งให้เป็นจุดเริ่มต้น การเขียนทำได้โดยลากเมาส์ออกจากจุดนี้และเมื่อลากกลับมาที่จุดนี้อีกครั้งหนึ่งก็จะถือว่าจบการเขียนตัวอักษรตัวนั้นและพร้อมที่จะเขียนตัวใหม่ต่อไปได้ ในเวลาต่อมาก็ได้มีการกำหนดขอบเขตของการเขียนโดยให้สามารถเขียนได้เฉพาะในพื้นที่ที่กำหนดเท่านั้น ถ้ามีการลากเมาส์ออกจากพื้นที่นี้ก็จะถือว่าจบการเขียนตัวอักษรตัวนั้นแล้วเช่นกันและเซทให้ mouse pointer กลับไปอยู่ที่จุดเริ่มต้นและพร้อมที่จะเขียนตัวใหม่ต่อไป การกำหนดขอบเขตนี้ก็ถือเป็นอีกตัวช่วยหนึ่งสำหรับการตัดตัวอักษรแทนที่จะต้องลากเมาส์กลับไปจุดเริ่มต้นเพียงอย่างเดียว จากรูปที่ 4.1 ประกอบ



รูปที่ 4.1 แสดงจุดเริ่มต้นและขอบเขตที่สามารถเขียนตัวอักษรได้

การเขียนโดยใช้วิธีนี้จำเป็นจะต้องมีกระบวนการในการตัดส่วนที่เกินมาซึ่งเป็นส่วนที่เกิดจากการลากเมาส์กลับไปจุดเริ่มต้นหรือเกิดจากการลากเมาส์ออกจากขอบเขตที่กำหนด รูปที่ 4.2 ประกอบ



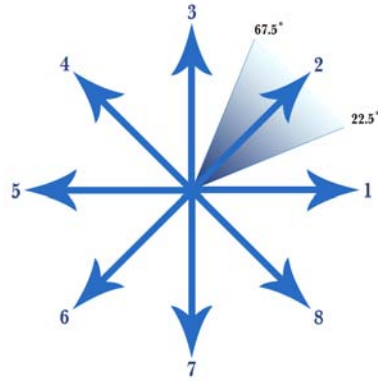
รูปที่ 4.2 แสดงส่วนของเส้นซึ่งไม่ต้องการและต้องตัดออก

วิธีการที่เราได้นำมาใช้สำหรับการตัดส่วนที่เกินมาก็คือ ใช้การพิจารณาจากทิศทางการเขียนว่ามีการเปลี่ยนทิศหรือไม่ ซึ่งจะเริ่มพิจารณาจากจุดท้ายสุดก่อนแล้วไล่ต่อไปเรื่อยๆ ถ้าทิศทางการเขียนเปลี่ยนตรงไหนก็จะเริ่มตัดตั้งแต่จุดนั้นไปจนถึงจุดสุดท้ายทิ้งไป (การเก็บข้อมูลของลายมือเขียน เบื้องต้นได้เก็บเป็นลิสของคู่อันดับ x กับ y) ดังนั้นจึงมีความจำเป็นที่ตัวอักษรที่เขียนขึ้นจะต้องมีมุมซึ่งเอาไว้บอกว่าเป็นจุดที่จะต้องตัดเสมอ ดังรูปที่ 4.3 ไม่เช่นนั้นแล้วการตัดตัวอักษรอาจเกิดความผิดพลาดได้



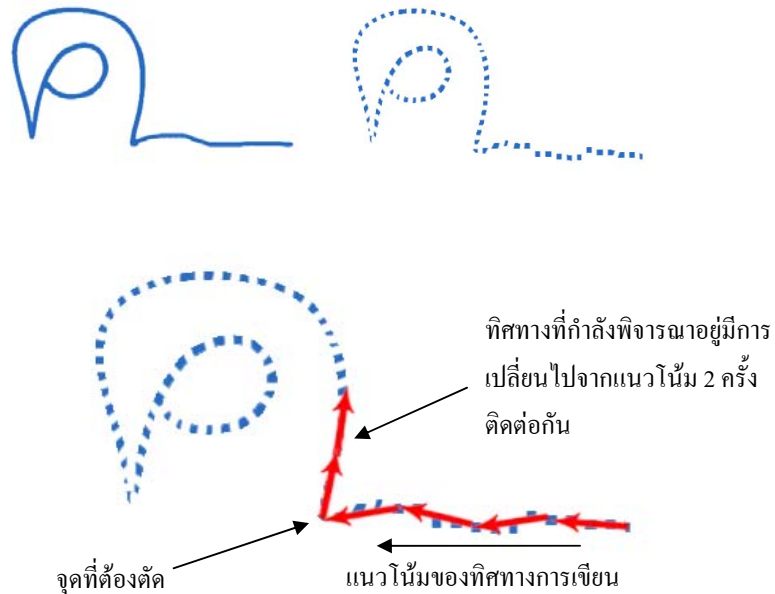
รูปที่ 4.3 แสดงตัวอักษรที่เขียนต้องมีมุมสำหรับตัดเสมอ

สำหรับทิศทางการเขียนเราได้แบ่งออกเป็น 8 ทิศทางแต่ละทิศทางครอบคลุมบริเวณใกล้เคียงรวม 45° ดังรูปที่ 4.4 ซึ่งทิศทางดังกล่าวหาได้จากการหามุมระหว่างจุดสองจุดเทียบกับแนวนอน โดยในที่นี้เราเลือกจุดที่อยู่ห่างกัน 4 ช่วงจุด (เช่น 1 กับ 5 และ 5 กับ 9 ต่อไปเรื่อยๆ) เมื่อได้มุมมาแล้วเราก็จะพิจารณาว่าอยู่ในทิศทางไหน



รูปที่ 4.4 แสดงทิศทางการเขียนที่เป็นไปได้

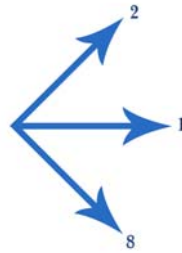
ในการที่จะตัดสินใจว่าการเปลี่ยนทิศทางการเขียนที่จุดไหน เราจะดูจากแนวโน้มของทิศทางว่าโดยรวมแล้วทิศทางกำลังไปทางไหนแล้วทิศทางที่กำลังพิจารณาอยู่มีการเปลี่ยนไปจากแนวโน้มหรือไม่ ถ้าทิศทางมีการเปลี่ยนไปจากแนวโน้ม (ในที่นี้เรากำหนดว่าต้องเปลี่ยนติดต่อกัน 2 ครั้ง) ก็จะถือว่าจุดนั้นเป็นจุดที่มีการเปลี่ยนทิศทาง ดังรูปที่ 4.5



รูปที่ 4.5 แสดงการพิจารณาตัดส่วนที่เกินมา

ในทางปฏิบัติเราได้พบว่า ในการลากเมาส์อย่างช้าๆ มีโอกาสเป็นไปได้สูงมากที่ทิศทางที่กำลังพิจารณาอยู่จะถูกตัดสินว่าได้เปลี่ยนทิศไปแล้วแต่ถ้าดูจากแนวโน้มแล้วมันไม่ได้เปลี่ยน ที่เป็นแบบนี้มีเนื่องมาจากการที่เราได้พิจารณาทิศทาง ทีละ 4 ช่วงจุด เมื่อมีการลากเมาส์อย่างช้าๆ จะทำให้จุดที่อ่านได้อยู่ชิดกันมากเมื่อนำมาหาทิศทางจึงทำให้ได้ทิศทางที่ผิดไปจากแนวโน้มมาก ดังนั้นเราจึงได้แก้ไขโดยการให้ทิศทางที่ทำให้เกิดการตีความว่าเกิดการเปลี่ยนทิศไปแล้ว

กลายเป็นว่ายังคงไม่มีการเปลี่ยนทิศ ดังรูปที่ 4.6 เมื่อแนวโน้มของทิศทางเป็น 1 ถ้าพบทิศ 2 กับ 8 ให้ถือว่าทิศทางยังไม่เปลี่ยน



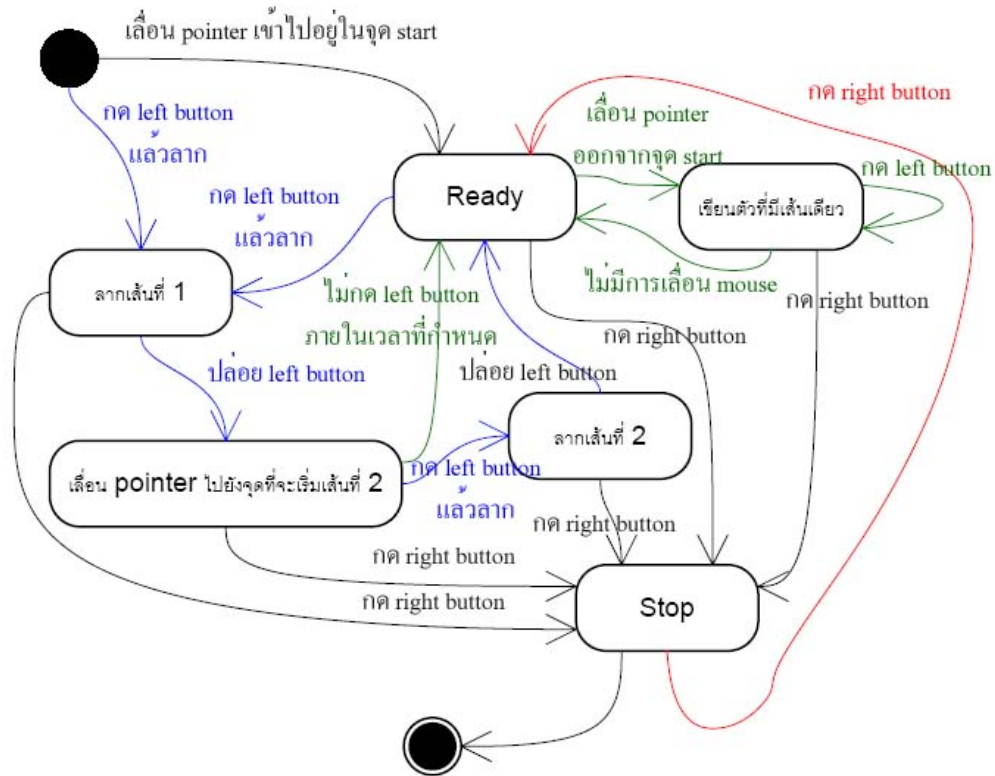
รูปที่ 4.6 เมื่อแนวโน้มของทิศทางเป็น 1 ถ้าพบทิศ 2 กับ 8 ให้ถือว่าทิศทางยังไม่เปลี่ยน

นอกจากวิธีการข้างต้นแล้ว เรายังได้มองหาวิธีการใหม่ที่เราคิดว่าน่าจะดีกว่า ซึ่งเราก็ได้นำเอา timer เข้ามาเป็นตัวช่วย โดยให้ถือว่าในขณะที่มีการลากเส้นถ้ามีการหยุดเลื่อนเมาส์ (ไม่มีการเลื่อนเมาส์เป็นช่วงเวลาหนึ่งซึ่งเท่ากับ timer ที่ตั้งไว้) ก็จะให้ mouse pointer กลับไปเริ่มที่จุดเริ่มต้นใหม่ทันที ซึ่งการใช้ timer นี้ได้ผลเป็นที่น่าพอใจมากที่สุดคือ สามารถเขียนได้อย่างรวดเร็วและมีความเป็นธรรมชาติมากกว่าวิธีแรกและที่สำคัญคือไม่มีส่วนที่ลากเกินมาซึ่งไม่ต้องการด้วย

อย่างไรก็ตามทั้งสองวิธีข้างต้นยังไม่สามารถใช้ได้กับตัวอักษรที่มีสองเส้น เช่น ส ศ ษ เป็นต้น (ตัวอักษรในภาษาไทยมีเพียง 1 เส้นกับ 2 เส้นเท่านั้น) จึงจำเป็นต้องหาวิธีการใหม่ที่สามารถใช้ได้กับตัวอักษรที่มีสองเส้นด้วย

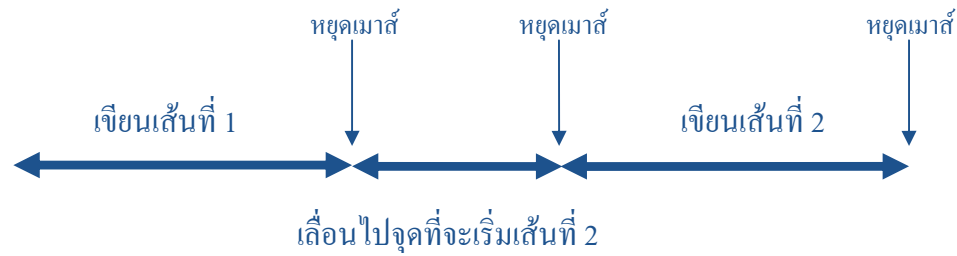
วิธีแรกที่เราได้คิดขึ้นมาเพื่อให้สามารถเขียนได้ทั้งตัวอักษรที่มีเส้นเดียวและตัวอักษรที่มีสองเส้นก็คือ ได้ยอมให้มีการคลิกเมาส์ได้เมื่อจะเขียนตัวอักษรที่มีสองเส้น ส่วนการเขียนตัวอักษรที่มีเส้นเดียวได้เลือกใช้วิธีเซต timer เพราะให้ผลที่ดีกว่า คือ เขียนได้เร็วและเป็นธรรมชาติมากกว่า

วิธีนี้มีข้อดี คือ สำหรับตัวอักษรที่มีเส้นเดียวจะสามารถเขียนได้อย่างรวดเร็วและเป็นรู้สึกเป็นธรรมชาติ แต่สำหรับการเขียนตัวอักษรที่มีสองเส้นซึ่งต้องใช้การคลิกเมาส์ช่วย ทำให้รู้สึกไม่สะดวกและยังทำให้เกิดความสับสนด้วยว่า เมื่อไหร่จะต้องคลิกเมาส์เมื่อไหร่ไม่ต้องคลิกเมาส์ และทำให้เขียนผิดได้บ่อยครั้ง รูปที่ 4.7 แสดง statechart ของการเขียนโดยใช้การเซต timer ร่วมกับการใช้คลิกเมาส์

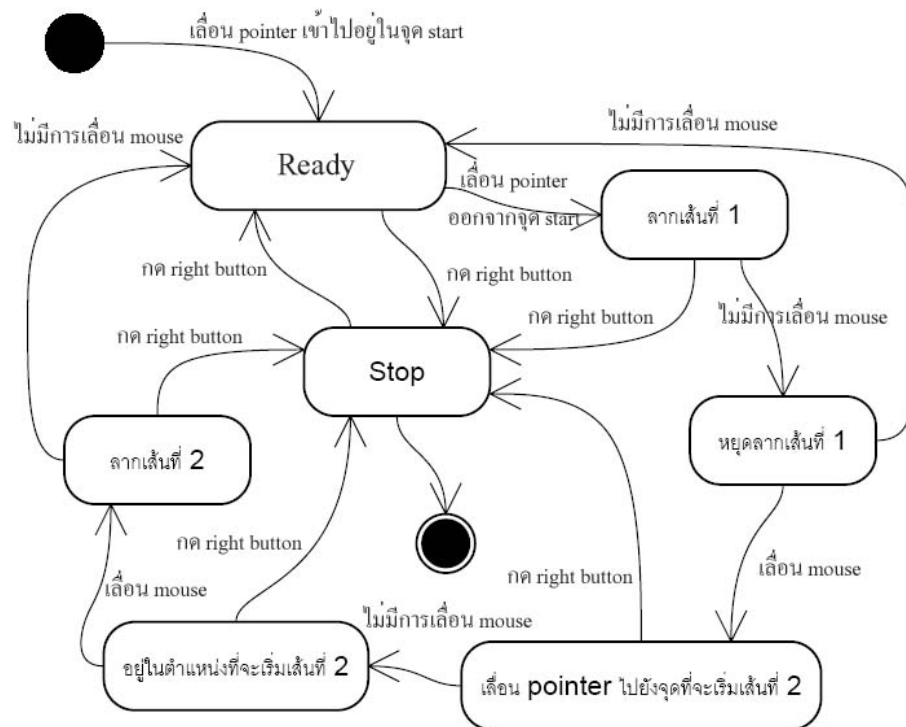


รูปที่ 4.7 แสดง statechart ของการเขียนโดยใช้การเซต timer ร่วมกับการใช้คลิกเมาส์

วิธีที่สองที่ได้คิดขึ้นมาเพื่อให้สามารถเขียนได้ทั้งตัวอักษรที่มีเส้นเดียวและตัวอักษรที่มีสองเส้นก็คือ เปลี่ยนจากการที่จะต้องคลิกเมาส์เมื่อจะเขียนตัวอักษรที่มีสองเส้นมาเป็นการใช้ timer ทั้งหมด โดยการเขียนตัวอักษรหนึ่งตัวจะถูกแบ่งออกเป็น 3 ช่วง คือ ช่วงที่หนึ่งสำหรับการลากเส้นที่ 1 และเมื่อมีการหยุดลากเมาส์ก็จะเข้าสู่ช่วงที่สองซึ่งจะทำให้เลื่อน mouse pointer ไปยังจุดที่จะเริ่มเส้นที่ 2 ถ้าไม่มีการเลื่อนเมาส์จนครบกำหนดเวลาในช่วงที่สองก็จะถือว่าจบการเขียนตัวอักษรตัวนั้นและ mouse pointer ก็จะกลับไปเริ่มที่จุดเริ่มต้นใหม่ซึ่งกรณีนี้จะเกิดขึ้นกับการเขียนตัวอักษรที่มีเส้นเดียว ดังรูปที่ 4.8 แต่ถ้ามีการเลื่อนเมาส์ในช่วงที่สองก็จะถือเป็นการเลื่อน mouse pointer ไปยังจุดที่จะเริ่มเส้นที่ 2 และเมื่อมีการหยุดเลื่อนเมาส์ก็จะเข้าสู่ช่วงที่สามซึ่งจะให้ลากเส้นที่ 2 เมื่อมีการหยุดลากเมาส์ก็จะถือว่าจบการเขียนตัวอักษรตัวนั้นและ mouse pointer ก็จะกลับไปเริ่มที่จุดเริ่มต้นใหม่ และใช้การเปลี่ยนรูปแบบ mouse pointer เพื่อเป็นการบอกว่าตอนไหนให้เขียนเส้นหรือตอนไหนให้เลื่อนเมาส์ไปอยู่ในตำแหน่งที่จะเขียนเส้นต่อไป รูปที่ 4.9 แสดง statechart ของการเขียนโดยใช้การเซต timer เพียงอย่างเดียว



รูปที่ 4.8 แสดงลำดับการเขียน



รูปที่ 4.9 แสดง statechart ของการเขียนโดยใช้การเซต timer เพียงอย่างเดียว

ข้อดีของวิธีที่สองนี้ก็คือ สามารถเขียนได้อย่างเป็นธรรมชาติโดยไม่ต้องอาศัยการคลิกเมาส์เข้าช่วยและสามารถใช้ได้กับตัวอักษรที่มีในภาษาไทยได้ครบทุกตัว แต่ก็มีข้อเสียคือทำให้การเขียนตัวอักษรที่มีเส้นเดียวซ้ำไปด้วย แต่ว่าวิธีนี้ไม่ได้เป็นปัญหาสำหรับโปรแกรมของเราเนื่องจากเราได้กำหนดตัวอักษรที่จะเขียนไว้ก่อนแล้วจึงให้ผู้ใช้เขียนตาม ดังนั้นเราจึงสามารถกำหนดได้ว่าตัวอักษรตัวนั้นควรจะใช้การกำหนดแบบไหน เช่น เรากำหนดให้เขียน ก เรารู้ว่า ก ต้องเขียนครั้งเดียว เราจึงกำหนดให้ใช้อัลกอริทึมสำหรับเขียนเส้นเดียวไปเลย แต่อย่างไรก็ตามสำหรับการนำไปใช้จริงอาจจะยังใช้งานได้ไม่ดีสำหรับตัวอักษรที่ต้องเขียนสองครั้ง

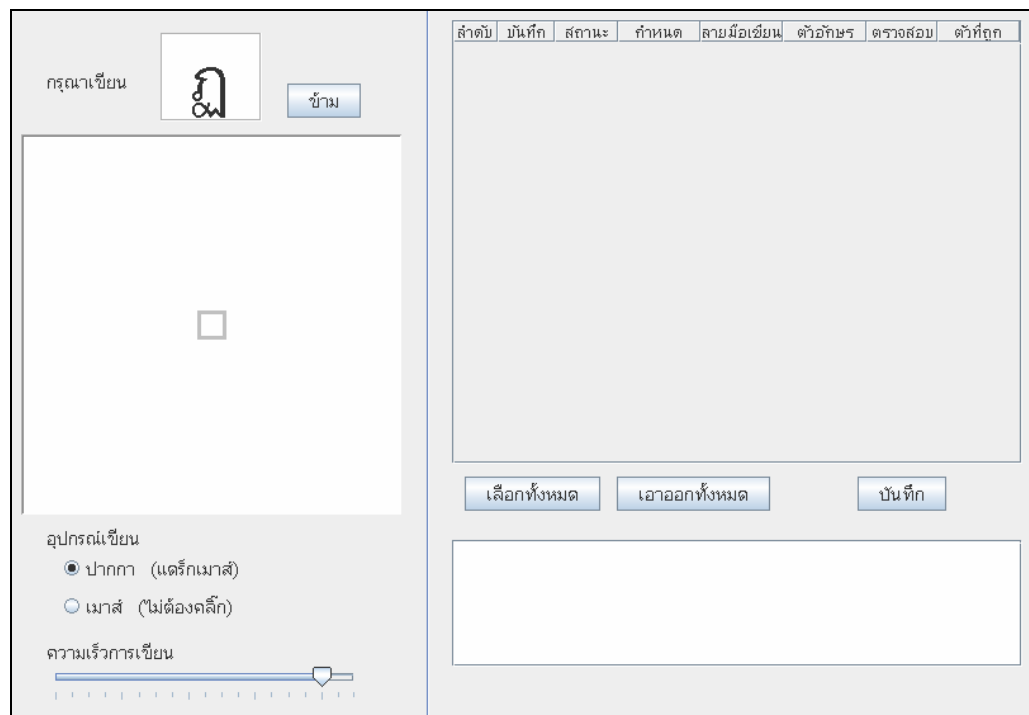
4.2 การออกแบบระบบ

ระบบเก็บฐานข้อมูลลายมือเขียนผ่านเว็บประกอบไปด้วยส่วนหลักๆ 3 ส่วน คือ

1. ส่วนรับลายมือเขียน (client)
2. server ซึ่งทำหน้าที่ recognize ลายมือเขียนและจับเก็บลายมือเขียนลงฐานข้อมูล
3. เว็บระบบจัดการฐานข้อมูลลายมือเขียน

4.2.1 ส่วนรับลายมือเขียน (client)

เป็นส่วนที่อินเทอร์เน็ตสำหรับรับลายมือเขียนแล้วทำการส่งลายมือเขียนนั้นไปให้ server ทำการ recognize แล้วส่งผลกลับคืนมาให้ผู้ใช้ตรวจสอบว่าถูกต้องหรือไม่ ผู้ใช้สามารถเลือกได้ว่าจะบันทึกลายมือลงในระบบหรือไม่ ส่วนรับลายมือเขียนนี้สามารถเรียกใช้โดยผ่านทางหน้าเว็บของระบบจัดเก็บฐานข้อมูลลายมือเขียน



รูปที่ 4.10 แสดง Java Applet สำหรับรับลายมือเขียน

สำหรับการพัฒนาส่วนนี้ได้พยายามหาวิธีที่ทำให้ผู้ใช้สามารถใช้งานได้สะดวก และไม่จำเป็นต้องใช้เพียงปากกาเท่านั้น ผู้ใช้สามารถใช้เมาส์ซึ่งเป็นอุปกรณ์ที่คอมพิวเตอร์ทุกเครื่องมีอยู่แล้วเขียนได้ โดยทุกๆ ไปแล้วการใช้เมาส์เขียนจะต้องอาศัยการคลิกเมาส์แล้วลากเพื่อให้ได้ตัวอักษรตามที่ต้องการ ซึ่งกลไกนี้เป็นกลไกเดียวกับการใช้ปากกาเขียนเพราะการคลิกเมาส์แล้ว

ลากก็เหมือนกับการกดปากกาแล้วลาก (เกิด event เดียวกัน) แต่ว่าการคลิกเมาส์แล้วลากทำให้รู้สึกไม่สะดวกเหมือนกับการเขียนด้วยปากกา จึงทำให้เกิดความต้องการหาวิธีที่ทำให้ไม่ต้องคลิกเมาส์ ดูหัวข้อที่ 4.1

4.2.2 Server

Server มีหน้าที่คอยรับการติดต่อจาก client เพื่อรับลายมือเขียนไปทำการ recognize แล้วส่งผลกลับไปให้ client และมีหน้าที่จัดการเก็บลายมือเขียนที่ส่งมาจาก client เมื่อผู้ใช้ได้ทำการตรวจสอบและเลือกบันทึกลายมือลงในระบบ

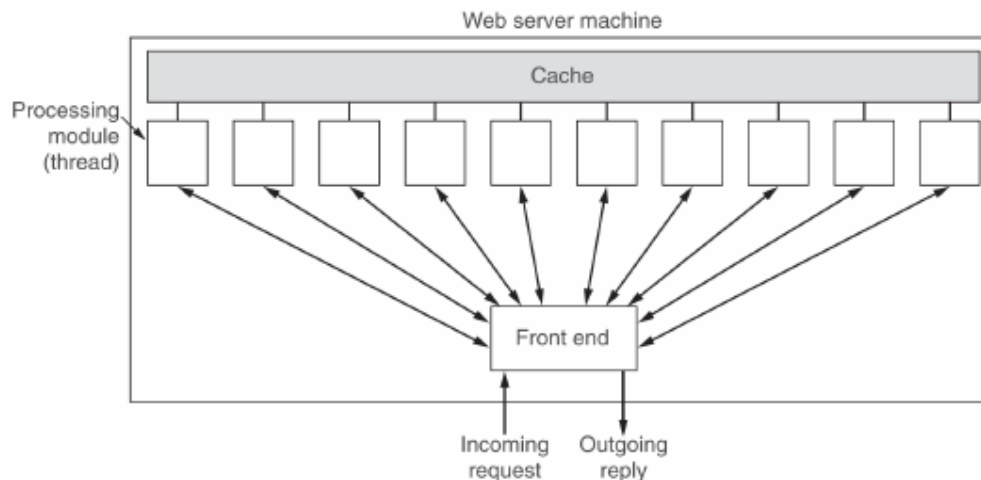
การพัฒนาในส่วนของเซิร์ฟเวอร์แอปพลิเคชัน ซึ่งใช้ในการติดต่อระหว่างผู้ใช้งานเว็บแอปพลิเคชันและในการติดต่อกับฐานข้อมูลเก็บลายมือเขียนนั้น พัฒนาโดยใช้เทคโนโลยีอินเทอร์เน็ต ซึ่งช่วยอำนวยความสะดวก ให้สามารถเรียกใช้อัลกอริทึมการรู้จำที่สร้างขึ้นมาจากภาษาต่างๆ ได้อย่างหลากหลายและได้โดยง่าย

สำหรับการทำงานนั้นมีการใช้เทคโนโลยีของซ็อกเก็ต ซึ่งเป็น TCP Software Interface มาตรฐานที่ใช้ในการติดต่อถึงกันผ่านระบบเครือข่าย โดยใน .Net นั้น การทำงานที่เกี่ยวข้องกับ Sockets จะต้องเรียกใช้งาน Namespace ที่ชื่อว่า System.Net.Sockets ซึ่งเป็น Namespace ที่บรรจุ Classes ต่างๆ ที่สนับสนุนการทำงานกับ Sockets โดย Class ที่เกี่ยวข้องกับการทำงานหลักๆ กับ Sockets เช่น NetworkStream, TcpClient, TcpListener, UdpClient, Socket เป็นต้น โดยคลาส Socket จะมีฟังก์ชันการทำงานพื้นฐานต่างๆ ที่จำเป็นสำหรับการสร้างแอปพลิเคชันที่ใช้งาน Socket สำหรับ Properties ที่สำคัญสำหรับ System.Net.Sockets.Socket เช่น

- AddressFamily ใช้ในการรับค่า address family ของ Socket
- LocalEndPoint ใช้ในการรับค่า end point ซึ่งเป็น network address ของ local
- ProtocolType ใช้ในการรับค่าชนิดของโปรโตคอลใน Socket
- SocketType ใช้ในการรับค่าชนิดของ Socket

นอกจากนี้ยังมี Method ที่สำคัญที่ใช้สำหรับการสร้างเซิร์ฟเวอร์แอปพลิเคชันอีก เช่น

- Accept ใช้สร้าง Socket ใหม่สำหรับการเชื่อมต่อที่เกิดขึ้น
- Bind ใช้ในการเชื่อมต่อ Socket เข้ากับ local endpoint
- Listen ใช้ในการให้ Socket รอรับการติดต่อจากไคลเอนท์
- Send ใช้ในการส่งข้อมูลให้กับ Socket ที่เชื่อมต่อกันอยู่
- Receive ใช้ในการรับข้อมูลจาก Socket ที่เชื่อมต่อกันอยู่
- Close ใช้ในการปิดการเชื่อมต่อกับ Socket และเคลียร์ทรัพยากรของระบบ



รูปที่ 4.11 แสดงการทำงานแบบมัลติเทรดในเซิร์ฟเวอร์

หลักการสำคัญอีกอย่างหนึ่งที่ใช้ในการพัฒนาเซิร์ฟเวอร์แอปพลิเคชันนี้คือ การทำงานแบบเป็นมัลติเทรด ซึ่งช่วยให้รองรับการใช้งานได้พร้อมกันโดยไม่มีปัญหา ซึ่งการสร้าง Thread สำหรับ .Net นั้น สามารถทำได้โดยเรียกใช้ Namespace ที่ชื่อ System.Threading โดยที่คลาส Thread นั้นจะเป็นตัวชี้ไปยังส่วนของโปรแกรมซึ่งจะทำงานบน Thread ใหม่ที่ถูกสร้างขึ้น สำหรับ method ที่สำคัญใน Thread Class ได้แก่ Start ใช้สำหรับให้ OS ทำการเปลี่ยนสถานะของ Thread ให้ทำงาน Abort ใช้ในการจบการทำงานของ Thread เป็นต้น นอกจาก Thread แล้ว ใน .Net ยังมี Namespace อีกตัวชื่อ ThreadPool ซึ่งเป็น Static Class โดยการทำงานนั้นจะเริ่มทำงานทันทีเมื่อเรียกใช้ ThreadPool โดยไม่ต้องสั่งให้ Start

นอกจากนี้แล้ว เซิร์ฟเวอร์แอปพลิเคชันนี้ สามารถทำการเพิ่มอัลกอริทึมในกระบวนการรู้จำเข้าไปได้ โดยการสร้างคลาสแบบ Managed ในการห่อหุ้ม Dynamic Link Library (DLL) ของการรู้จำ เพื่อเรียกใช้อัลกอริทึมในการรู้จำผ่าน .NET Interoperation Services ซึ่งช่วยให้การทำงานระหว่างภาษาต่างชนิดกัน โดยช่วยให้ .NET สามารถทำงานร่วมกันโค้ดอื่นๆ ที่อยู่นอกจาก .NET Environment ทำได้โดยง่าย เช่น C# สามารถเรียกใช้คลาสจาก Unmanaged C++ ได้โดยตรง สำหรับขั้นตอนการเพิ่มอัลกอริทึมในการรู้จำ มีขั้นตอนหลัก 3 ส่วน ดังต่อไปนี้

ส่วนแรก ในส่วนของการรู้จำ ทำการสร้างไฟล์ .Dll ขึ้นมา จากโค้ดการรู้จำตัวอักษร อัลกอริทึมแบบต่างๆ ที่มีอยู่

ส่วนที่สอง ในส่วนของคลาสที่ใช้ห่อหุ้มไลบรารีที่ได้สร้างไว้ (Wrapper Class) เนื่องจากไฟล์ .Dll ที่ได้จาก .NET มีสถาปัตยกรรมแตกต่างจากอย่างอื่น โดย .NET จะสร้างไฟล์คอมโพเนนต์ในรูปแบบของตัวเอง เรียกกันตรงๆ ว่า .NET component แม้ว่าผลลัพธ์คือไฟล์นามสกุล .DLL เหมือนกัน แต่ภายในมีโครงสร้างที่ต่างกันอย่างสิ้นเชิง การทำให้สามารถเรียกใช้โค้ด

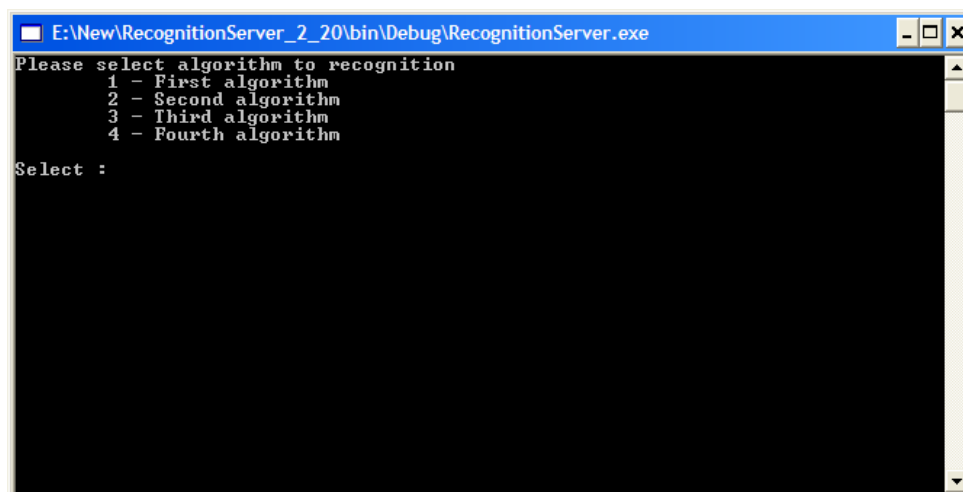
Unmanaged DLL จาก .NET component เรียกว่าการทำ Interop โดย Wrapper Class ที่สร้างขึ้นมานี้ จะทำให้ .NET มองเห็นไฟล์ .DLL เสมือนเป็น .NET Class ทั่วไป

ส่วนที่สาม ในส่วนโปรแกรมหลักของเซิร์ฟเวอร์ ซึ่งในที่นี้เขียนโดย C# ซึ่งเป็น Managed Extension (การเขียนโปรแกรมใน Managed Environment ซึ่งเป็นส่วนหนึ่งของ .NET Framework) สามารถเรียกใช้งานไลบรารีของการรู้จำโดยโดยตรง ผ่าน Wrapper Class ที่สร้างขึ้น

4.2.2.1 การทำงานของเซิร์ฟเวอร์แอปพลิเคชัน

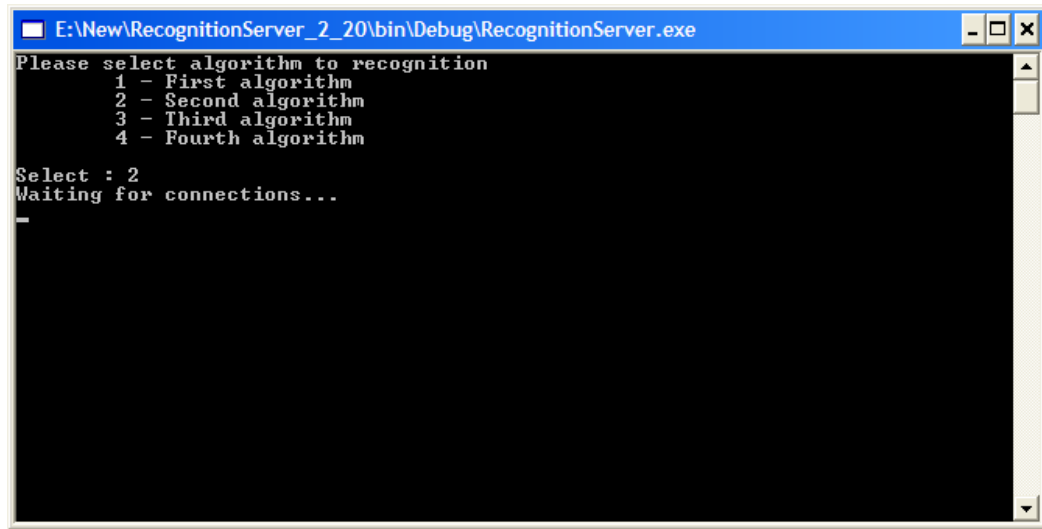
เซิร์ฟเวอร์แอปพลิเคชันทำงานในโหมดคอนโซล โดยเมื่อเริ่มต้นเปิดใช้งาน ผู้ใช้จะพบกับหน้าจอให้เลือกอัลกอริทึมที่จะใช้ในการรู้จำ ซึ่งในขั้นตอนนี้มีเพียงอัลกอริทึมเดียว (โดยได้อธิบายวิธีการในการเพิ่มอัลกอริทึมให้ผู้ที่ต้องการใช้ในหัวข้อก่อนหน้านี้แล้ว) หลังจากเลือกอัลกอริทึมเรียบร้อยแล้ว จะเริ่มเข้าสู่กระบวนการเทรนข้อมูล เพื่อเตรียมไว้ใช้ในกระบวนการรู้จำตัวอักษร ต่อมาเซิร์ฟเวอร์จะทำการเปิดการเชื่อมต่อและรอรับการเชื่อมต่อจากไคลเอนท์ โดยเป็นแบบมัลติเทรต เพื่อให้สามารถรองรับการทำงานของผู้ใช้ได้หลายคนพร้อมกัน ซึ่งตัวไคลเอนท์นี้ เป็นเว็บแอปพลิเคชัน ที่ทำหน้าที่รับและจัดการข้อมูลลายมือเขียนจากผู้ใช้ ผ่านทางเว็บเบราว์เซอร์ และจัดส่งให้กับเซิร์ฟเวอร์

หลังจากเซิร์ฟเวอร์ได้รับข้อมูลลายมือเขียนกลับมาแล้ว ก็จะทำการส่งข้อมูลนั้นเข้าสู่กระบวนการรู้จำ และส่งค่าตัวอักษรที่ได้จากการรู้จำนั้นกลับคืน เพื่อแสดงผลการทำงานสู่ผู้ใช้ โดยที่ผู้ใช้สามารถจัดการกับข้อมูลลายมือเขียนของตนเองได้ และยังสามารถจัดเก็บลายมือเขียนที่สามารถรู้จำได้อย่างถูกต้องนั้น เพื่อเก็บไว้ใช้ประโยชน์ต่อไป



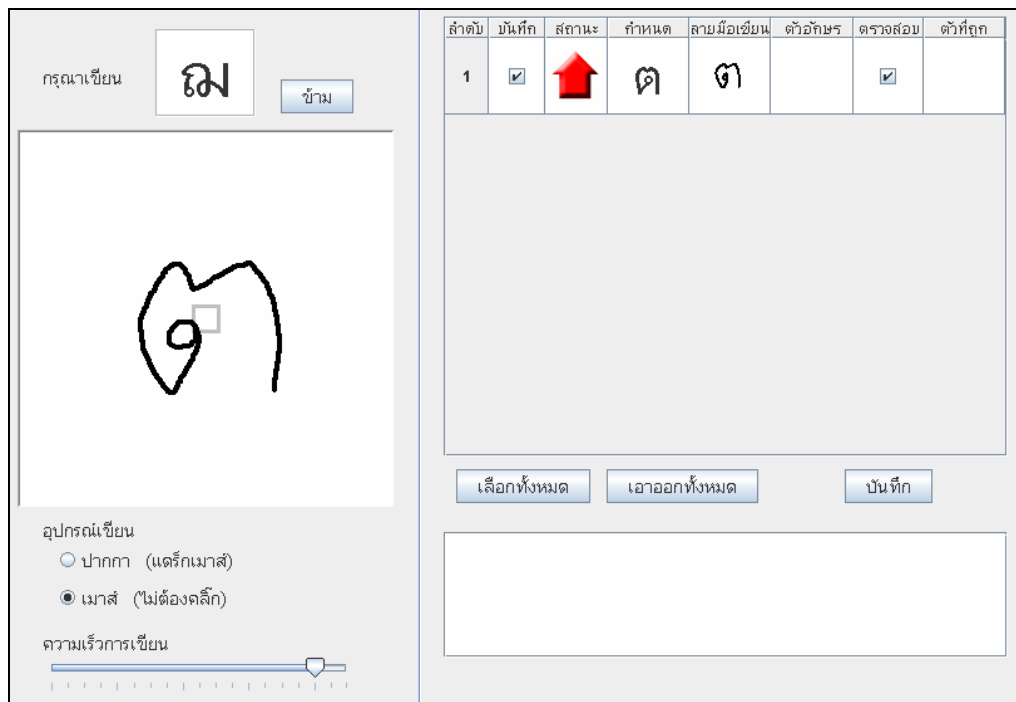
รูปที่ 4.12 แสดงเซิร์ฟเวอร์ขณะรอให้ผู้ใช้อ้อนอัลกอริทึมที่ต้องการใช้ในกระบวนการรู้จำ

เมื่อผู้ใช้เลือกอัลกอริทึมเรียบร้อยแล้ว เซิร์ฟเวอร์จะทำการเทรนข้อมูลเพื่อเตรียมไว้ใช้ในกระบวนการรู้จำ หลังจากนั้นจึงเปิดรอรับการเชื่อมต่อจากไคลเอนต์แบบมัลติเทรต ดังรูปที่ 4.13



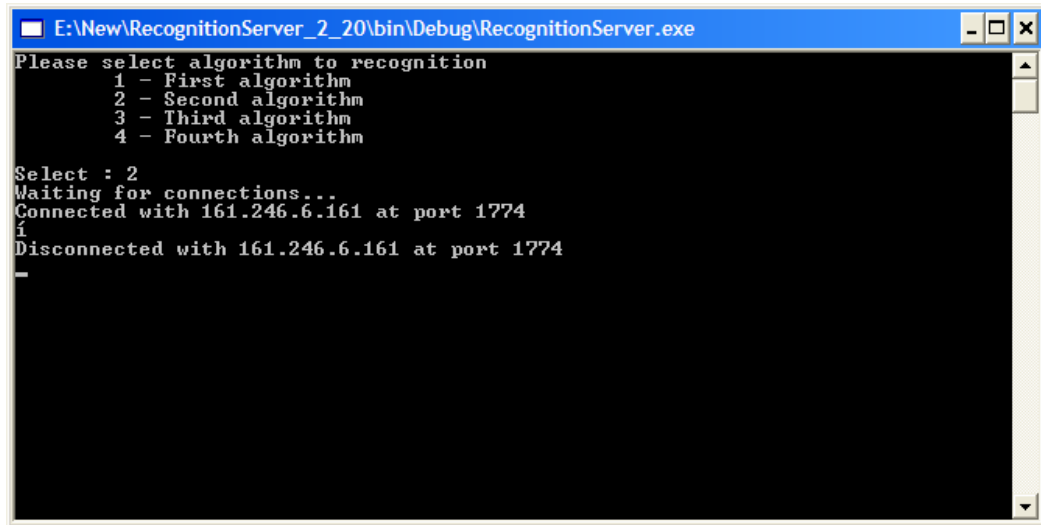
รูปที่ 4.13 แสดงการรอรับการเชื่อมต่อจากไคลเอนต์ของเซิร์ฟเวอร์

ที่โปรแกรมไคลเอนต์ เมื่อผู้ใช้ป้อนข้อมูลลายมือเขียนเข้ามา โดยการลากเมาส์เป็นตัวอักษร ไคลเอนต์จะติดต่อเพื่อขอรับบริการจากเซิร์ฟเวอร์ โดยจะส่งข้อมูลลายมือเขียนมาให้เซิร์ฟเวอร์นำเข้าสู่กระบวนการรู้จำต่อไป



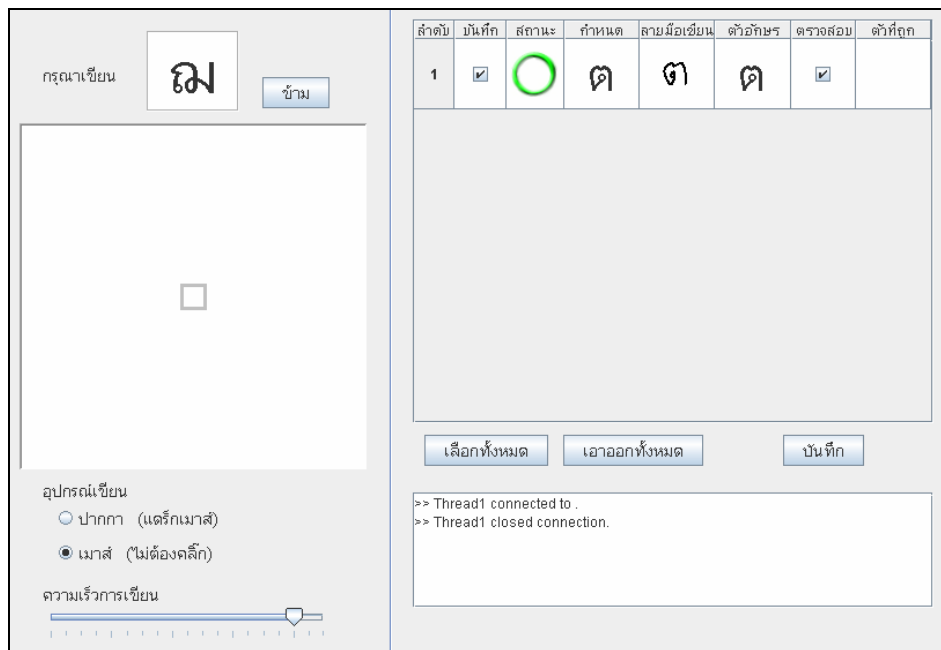
รูปที่ 4.14 แสดงการเขียนลายมือ

จากนั้น เมื่อเซิร์ฟเวอร์ได้รับข้อมูล และตรวจสอบเรียบร้อยแล้ว เซิร์ฟเวอร์จะส่งผลลัพธ์ที่ได้จากกระบวนการรู้จำไปให้ไคลเอนท์ และปิดการเชื่อมต่อ ดังรูปที่ 4.15



รูปที่ 4.15 แสดงการปิดการเชื่อมต่อของเซิร์ฟเวอร์

ที่ไคลเอนท์ จะแสดงผลลัพธ์ที่ได้รับจากเซิร์ฟเวอร์ เพื่อให้ผู้ใช้ตรวจสอบว่าได้ผลถูกต้องหรือไม่ ดังรูปที่ 4.16



รูปที่ 4.16 แสดงผลการรู้จำลายมือเขียน

เมื่อผู้ใช้งานต้องการจัดเก็บข้อมูลลายมือเขียนของตนไว้ จะทำการเลือกข้อมูลนั้นๆ และคลิกที่บันทึกลายมือ โดยมีการตรวจสอบความถูกต้องของข้อมูลที่ได้รับมาก่อนด้วย จากนั้นจึงจะส่งข้อมูลเหล่านี้ไปให้เซิร์ฟเวอร์

เมื่อเซิร์ฟเวอร์ได้รับข้อมูลที่ต้องการให้มีการจัดเก็บ จะทำการจัดเก็บข้อมูลลายมือเขียนเหล่านั้นเข้าสู่ฐานข้อมูล โดยจะแสดงจำนวนข้อมูลที่ผ่านมาการตรวจสอบจากผู้ใช้งานแล้วว่ามี ความถูกต้องและผิดพลาดเป็นจำนวนเท่าไร และเก็บข้อมูลเหล่านี้ไว้เป็นสถิติเพื่อใช้งานต่อไปได้

4.3 เว็บระบบจัดเก็บฐานข้อมูลลายมือเขียน

เป็นเว็บสำหรับให้ผู้ดูแลระบบและผู้ใช้งานมาใช้ระบบจัดเก็บฐานข้อมูลลายมือเขียนได้ โดยมีคุณสมบัติของหน้าเว็บสำหรับผู้ดูแลระบบและผู้ใช้งานแตกต่างกัน และเมื่อจะเข้าไปใช้งานในระบบผู้ใช้งานจะต้องทำการป้อนชื่อผู้ใช้งานและรหัสผ่านก่อน ดังรูปที่ 4.17 แต่ถ้าผู้ใช้งานลืมรหัสผ่านก็ให้คลิกลิงค์สำหรับผู้ลืมรหัสผ่าน ซึ่งในหน้านี้จะมีช่องให้ใส่ชื่อผู้ใช้งานแล้วระบบจะทำการส่งรหัสผ่านไปให้ทางอีเมลแอดเดรสของผู้ใช้

รูปที่ 4.17 แสดงหน้าล็อกอิน

4.3.1 สำหรับผู้ดูแลระบบ

เมื่อเข้ามาสู่หน้าแรกจะมีตารางแสดงจำนวนลายมือที่ได้ทำการทดสอบการรู้จำแยกตามอัลกอริทึมที่ใช้ในการทดสอบ ดังรูปที่ 4.18

สถิติในการรู้จำ			
อัลกอริทึม	ตัวอักษรที่ถูกต้อง	ตัวอักษรที่ผิดพลาด	เปอร์เซ็นต์ความถูกต้อง
1	34	4	89 เปอร์เซ็นต์
2	53	13	80 เปอร์เซ็นต์
3	20	10	66 เปอร์เซ็นต์
4	48	3	94 เปอร์เซ็นต์

รูปที่ 4.18 แสดงสถิติการรู้จำลายมือเขียน

มีส่วนสำหรับให้ผู้ดูแลระบบใช้ในการเพิ่ม- ถอนรายชื่อผู้ใช้หรือแก้ไขข้อมูลของผู้ใช้ เนื่องจากผู้ดูแลระบบเป็นผู้ที่มีหน้าที่ในการเพิ่ม- ถอนรายชื่อผู้ใช้เข้าไปในระบบ ผู้ใช้คือคนที่ถูกเลือกให้เข้ามาใช้ระบบได้เท่านั้น ไม่มีหน้าเว็บสำหรับให้ผู้ใช้สมัครเข้าเป็นสมาชิก ดังรูปที่ 4.19 และ 4.20

หน้าแรก	แสดงรายชื่อผู้ใช้	เพิ่มผู้ใช้	แสดงลายมือทั้งหมด	แจ้งข่าวสาร
---------	-------------------	-------------	-------------------	-------------

ชื่อผู้ใช้

takemura

รหัสผ่าน

123456

อีเมล

takemura@nh.jp

บันทึก

รูปที่ 4.19 แสดงหน้าเพิ่มผู้ใช้

Buttons: หน้าแรก, แสดงรายชื่อผู้ใช้, เพิ่มผู้ใช้, แสดงลายมือทั้งหมด, แจ้งข่าวสาร

	ชื่อผู้ใช้	หน้าที่	อีเมล
☐	setsuna	admin	s5010049@kmitl.ac.th
☐	bbb	user	s5010049@kmitl.ac.th
☐	ccc	user	s5010049@kmitl.ac.th
☐	ddd	user	s5010049@kmitl.ac.th
☐	ggg	user	s5010049@kmitl.ac.th
☐	keuchirou	user	s5010049@kmitl.ac.th

ลบผู้ใช้

รูปที่ 4.20 แสดงหน้าแสดงรายชื่อผู้ใช้

มีหน้าเว็บสำหรับให้ผู้ใช้และระบบสามารถส่งอีเมลไปยังผู้ใช้ได้ เมื่อมีข่าวสารที่ต้องการแจ้งให้ผู้ใช้ทราบ

Buttons: หน้าแรก, แสดงรายชื่อผู้ใช้, เพิ่มผู้ใช้, แสดงลายมือทั้งหมด, แจ้งข่าวสาร

จาก : admin@setsunakami.kmitl.ac.th

ถึง : ถึงผู้ใช้ทุกคน

เรื่อง :

ส่งข่าว

รูปที่ 4.21 แสดงหน้าสำหรับส่งอีเมลเพื่อแจ้งข่าวสาร

ผู้ดูแลระบบสามารถเข้าไปดูลายมือเขียนของผู้ใช้ได้ และสามารถลบข้อมูลลายมือเขียนของผู้ใช้ได้ (ในกรณีที่มีข้อมูลลายมือเขียนที่ไม่ถูกต้อง)

4.3.2 สำหรับผู้ใช้

มีส่วนสำหรับผู้ใช้มีส่วนที่ใช้สำหรับเขียนลายมือและส่วนสำหรับเข้าไปดูลายมือเขียนของตนเองและสามารถลบข้อมูลลายมือเขียนของตนเองได้ (ในกรณีที่มีข้อมูลลายมือเขียนที่ไม่ถูกต้อง) ดังรูปที่ 4.22

รูปที่ 4.22 แสดงหน้าตรวจสอบลายมือเขียน

สามารถดาวน์โหลดข้อมูลลายมือเขียนทั้งของตนเองและทั้งฐานข้อมูลไปใช้ได้ โดยข้อมูลจะถูกดึงมาจากฐานข้อมูลแล้วบันทึกเป็นเท็กไฟล์ ดังรูปที่ 4.23

```

character.txt - Notepad
File Edit Format View Help
#
n=0
$xy
(157,133)(162,123)(166,113)(169,103)(172,94)(172,90)(173,87)(173,85)(173,84)(170,83)(167,82)
(162,82)(159,80)(157,79)(156,79)(156,78)(156,76)(156,74)(156,72)(160,71)(164,69)(168,68)(173,67)
(179,67)(186,67)(192,67)(198,67)(202,67)(205,69)(209,71)(211,74)(212,78)(214,83)(215,89)(215,95)
(216,103)(216,111)(216,120)(216,130)(216,137)(216,145)(214,151)(214,155)(212,158)(212,162)
(211,162)
#
n=1
$xy
(146,136)(147,130)(148,123)(149,117)(151,111)(151,107)(151,103)(151,101)(151,100)(150,100)
(148,100)(145,100)(142,100)(138,100)(133,100)(129,100)(125,100)(121,100)(119,100)(118,100)
(118,99)(118,97)(120,94)(123,91)(126,87)(131,84)(136,81)(140,79)(142,77)(143,77)(144,77)(145,77)
(147,77)(148,77)(150,77)(153,77)(157,77)(162,77)(166,77)(170,79)(174,81)(177,83)(179,86)(181,89)
(182,94)(182,100)(182,104)(182,109)(182,115)(182,120)(182,125)(182,129)(182,134)(182,138)
(182,141)(182,144)(182,146)(182,147)
#
n=2
$xy
(145,161)(145,160)(145,159)(145,158)(145,157)(145,156)(145,155)(145,154)(145,152)(145,150)
(146,146)(146,142)(146,136)(146,127)(146,117)(146,108)(146,98)(146,90)(146,85)(146,83)(145,83)
(142,83)(138,83)(131,83)(125,83)(119,84)(114,84)(111,85)(111,84)(112,81)(116,78)(122,72)(131,67)
(139,61)(148,55)(156,51)(165,48)(173,44)(179,43)(185,43)(191,43)(194,43)(197,43)(199,45)(200,48)

```

รูปที่ 4.23 แสดงไฟล์ลายมือเขียน